

# Ignition Perspective

*A hands-on curriculum for Perspective Designer*

---

## Ignition **8.3.9** · Inductive Automation

How Perspective actually works — the Gateway/Designer/Session model, views and containers, bindings and transforms, tags and UDTs, scripting, security and alarms — followed by a seven-day training plan, eighteen hands-on labs, and a capstone project.

---

**18** labs    **7**-day plan    **1** capstone project    **18** reference parts

Prepared September 15, 2026

# Table of Contents

---

How to Use This Guide ..... 3

Part 1. The Mental Model ..... 4

Part 2. Designer Tour ..... 7

Part 3. Views, Containers, and Layout ..... 9

Part 4. Properties and Bindings ..... 12

Part 5. Events, Actions, and Scripting ..... 15

Part 6. Data: Tags, History, and SQL ..... 19

Part 7. Pages, Docks, Popups, and Navigation ..... 21

Part 8. Styling and Themes ..... 23

Part 9. Security ..... 24

Part 10. Alarms ..... 25

Part 11. What Is New in 8.3 ..... 26

Part 12. The Training Plan ..... 27

Part 13. Labs ..... 31

Part 14. Troubleshooting Playbook ..... 44

Part 15. Veteran Rules ..... 45

Part 16. Cheat Sheets ..... 46

Part 17. Glossary ..... 49

Part 18. Self Assessment ..... 51

# How to Use This Guide

---

Three passes.

- 1. **Skim Parts 1–3 (30 min).** Do not try to memorize. You are building vocabulary so the Designer stops looking like noise.
- 2. **Work Part 12's plan (about 7 sessions of 2–3 hours).** Each day points at labs in Part 13. Type every step. Do not copy-paste blindly — you learn the Designer through your hands.
- 3. **Keep Parts 14–17 open while you build.** Troubleshooting, rules, cheat sheets, glossary.

## Conventions in this document

| Notation                                                                          | Meaning                                    |
|-----------------------------------------------------------------------------------|--------------------------------------------|
| <code>Project Browser &gt; Perspective &gt; Views</code>                          | A navigation path in the Designer          |
| <code>[default]Line1/Tank1/Level</code>                                           | A tag path (provider in brackets)          |
| <code>props.text</code>                                                           | A property path on a Perspective component |
|  | A place beginners reliably get hurt        |
|  | The idea that makes the section click      |

**Verify as you go.** This guide is accurate to the 8.3 line as I know it, but module availability differs by install and point releases move things. When something here doesn't match your screen, the manual at `docs.inductiveautomation.com` (switch the version selector to **8.3**) and the free videos at `inductiveuniversity.com` are the authority. Treat a mismatch as a chance to learn where the real documentation lives, not as a blocker.

# Part 1. The Mental Model

If you get this part wrong, everything else feels arbitrary. If you get it right, most of Perspective is guessable.

## 1.1 Three things called "Ignition"

**The Gateway** is the server. It is a Java application running as a service on some machine. It:

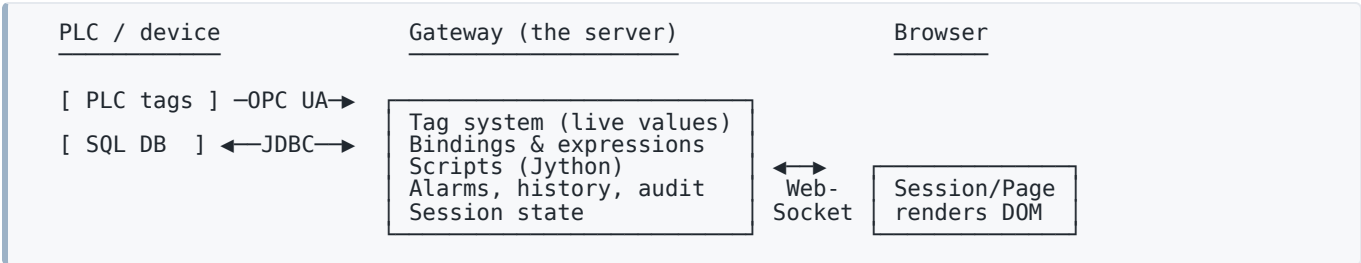
- connects **down** to equipment (OPC UA to PLCs, Modbus, drivers, MQTT),
- connects **sideways** to databases (SQL Server, Postgres, MySQL...),
- holds the **tag system** — the live, in-memory picture of your plant,
- **executes all project logic** (bindings, scripts, queries, alarms),
- and **serves** the web pages and web APIs.

You administer it in a browser, usually `http://<gateway-host>:8088` (or `8043` for HTTPS).

**The Designer** is a desktop app you launch from the Designer Launcher. It connects to the Gateway and edits projects **live on the server**. There is no "deploy" step in the classic sense — when you press save, the running Gateway picks up the change and open sessions update.

**A Perspective Session** is your project running in a web browser (or the Perspective mobile app).

 **The single most important sentence in this guide:** in Perspective, the browser is a *renderer*. The Gateway holds the state and runs the logic. The two talk over a persistent WebSocket. When you click a button, the click travels to the Gateway, the Gateway runs your script and mutates properties, and the changed properties travel back down and the browser re-renders.



**Why this matters immediately:**

- Your Python scripts run **on the Gateway**, not in the browser. You cannot `print` to a browser console and you cannot open a file on the operator's laptop from a script.
- Every bound property is a subscription the Gateway maintains for that session. 500 operators with a heavy screen open is 500× the work. Screen design is a performance decision.
- A session survives a flaky network better than you expect, but it is not offline-capable by default (8.3 adds specific offline form support — see [Part 11](#)).

## 1.2 Perspective vs Vision

Your Gateway probably has both modules. Know which world you're in.

|                 | Perspective                                        | Vision                                         |
|-----------------|----------------------------------------------------|------------------------------------------------|
| Runs in         | Web browser / mobile app                           | Java desktop client (launcher)                 |
| Layout          | CSS-like: flex, breakpoints, responsive            | Fixed pixel canvas with anchoring              |
| Scripting scope | <b>Gateway only</b>                                | Client-scoped (runs on the operator's machine) |
| Unit of design  | <b>View</b>                                        | <b>Window</b>                                  |
| Mobile          | First class                                        | No                                             |
| Best for        | New work, phones/tablets, anything customer-facing | Legacy plants, existing Vision investment      |

If you are learning today, learn Perspective. Vision is supported but Perspective is where new development goes.

## 1.3 Projects and resources

A **project** is a named container of resources living on the Gateway:

- Perspective **Views**, **Page Configuration**, **Session Properties**, **Styles**
- **Named Queries** (parameterized SQL)
- **Scripting > Project Library** (your shared Python modules)
- **Gateway Events** and **Session Events** (scheduled/triggered scripts)
- Vision windows and templates, if you use Vision
- In 8.3, also **Event Streams** and **Drawing** resources

**Inheritance.** A project can have a **parent project**. Children inherit and can override the parent's resources. The standard pattern in a real plant:

```
"Global"      ← styles, common views, shared script library, named queries
├── "Plant_A" ← inherits Global, adds plant-specific screens
└── "Plant_B"
```

Start a *single* project for training. Learn inheritance later, but know it exists — it is how professional shops avoid copy-pasting a header bar into nine projects.

**8.3 stores project resources as JSON files on disk**, under the Gateway's data directory. Each resource is a small folder with a `resource.json` plus its payload (a Perspective view is a `view.json`). That makes projects diff-able and genuinely Git-friendly — one of the headline reasons shops upgrade to 8.3. You will not edit these by hand as a beginner, but knowing your views are plain JSON explains why copy-paste between Designers works, and why "export/import" is cheap.

## 1.4 Tags: the noun of the whole system

A **tag** is a named value in the Gateway with metadata: a value, a **quality**, a **timestamp**, a datatype, engineering units, alarms, and history settings.

Tags live in a **tag provider**. Out of the box you have `default` (your real tags) and `System` (diagnostics like `[System]Gateway/CurrentDateTime`). Tag paths look like:

```
[default]Line1/Tank1/Level
[System]Gateway/Performance/CPU Usage
```

### Tag types you will meet:

| Type           | What it is                                            | Typical use                                   |
|----------------|-------------------------------------------------------|-----------------------------------------------|
| OPC            | Bound to a device address through a device connection | The real PLC value                            |
| Memory         | Value lives only in Ignition                          | Setpoints, simulation, scratch values         |
| Expression     | Computed from an expression, re-evaluates on change   | <code>{[.]Level} / {[.]Capacity} * 100</code> |
| Query          | Computed by running SQL on a poll interval            | Slow-moving business data                     |
| Derived        | Read/write transform over a source tag                | Unit conversion with writeback                |
| Reference      | A pointer to another tag                              | Aliasing, tag reorganization                  |
| UDT Definition | A <i>type</i> : a reusable tag structure              | "Motor", "Tank", "Valve"                      |
| UDT Instance   | An <i>instance</i> of that type                       | <code>Line1/Pump3</code> of type "Motor"      |

 **UDTs are the single biggest leverage point in Ignition.** Define "Motor" once with Running, Fault, Speed, Hours, its alarms and its history settings — then stamp 200 instances. Change the definition, all 200 update. Combined with a Perspective view that takes a tag path as a parameter, you build *one* motor faceplate and reuse it everywhere. Every experienced Ignition developer thinks in UDTs. Learn them early (Lab 8).

**Quality matters.** A tag value is never just a number; it carries quality (`Good`, `Bad_NotFound`, `Bad_Stale`, `Uncertain`...). Perspective will draw a **quality overlay** on components bound to a bad tag — that diagonal-hatch or red marker you'll eventually see is Ignition telling you the truth about your data, not a bug. Never suppress it globally.

## 1.5 The lifecycle of a click

Commit this to memory; it explains nearly every "why isn't this working" question.

1. Operator clicks a Button in the browser.
2. The browser sends an event over the WebSocket to the Gateway.
3. The Gateway runs your `onActionPerformed` script **in Jython, on the server**.
4. Your script writes a tag with `system.tag.writeBlocking(...)`.
5. The tag system pushes the value to the PLC via the device connection.
6. The PLC value changes; the tag subscription fires.
7. Every binding on that tag, in every open session, recomputes.
8. Changed properties stream down the WebSocket; browsers re-render.

Steps 4–7 are asynchronous and can take a round trip. △ This is why "I wrote the tag and read it back on the next line and got the old value" is the classic beginner bug. Don't write-then-read; write, and let the binding update the screen.

# Part 2. Designer Tour

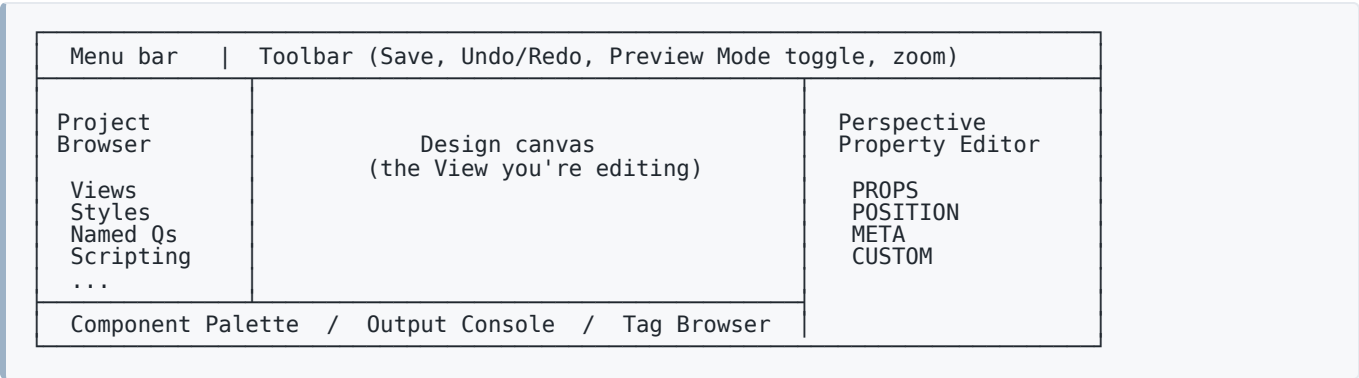
## 2.1 Getting in

1. From the Gateway home page, download and install the **Designer Launcher** (once per machine).
2. Add your Gateway by hostname/IP. It appears as a tile.
3. Launch, log in with a user that has the `Designer` role, and pick your project.

**Designer sessions are collaborative.** Multiple people can be in the same project. Perspective resources get **locked** by whoever opens them for editing — if you see a resource marked as in use by another user, that's the concurrency system, not a fault.

## 2.2 The Perspective workspace, panel by panel

When you open a View, the Designer arranges itself like this:



**Project Browser (left).** The tree of everything in the project. `Perspective > Views` is where you live. Right-click folders to create new views. When a view is open, the browser also shows the **component hierarchy** of that view — this is your reliable way to select a component you can't click on the canvas (small, hidden, or underneath something).

**Design canvas (center).** Drag, drop, resize. Two modes, toggled in the toolbar:

- **Design mode** — clicks select components for editing.
- **Preview mode** — clicks behave like a real session: buttons fire, bindings live, navigation works. Get in the habit of flipping to Preview constantly. Half of "it doesn't work" is "you were still in design mode."

**Perspective Property Editor (right).** The most important panel in the product. Four tabs:

| Tab      | Contains                                                                                                                              | Think of it as                                            |
|----------|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| PROPS    | Component-specific properties: <code>text</code> , <code>value</code> , <code>style</code> , <code>columns</code> , <code>data</code> | <i>What this component is</i>                             |
| POSITION | How the component sits <b>in its parent container</b>                                                                                 | <i>Where it goes</i> — meaning changes per container type |
| META     | <code>name</code> , <code>visible</code> , <code>tooltip</code> , <code>domId</code> , <code>enabled</code> hints                     | <i>Identity and visibility</i>                            |
| CUSTOM   | Properties <b>you invent</b>                                                                                                          | <i>Your view's variables / local state</i>                |



**CUSTOM is not an afterthought — it's your architecture.** Professional Perspective views put their state in custom properties on the root container, bind components to those, and never let components reach sideways into each other. More on this in [4.6](#).

**Component Palette (bottom or right).** Categorized: Containers, Display, Input, Charts, Tables, Alarming, Symbols, Embedding, Navigation, Reporting. Search it — it's faster than browsing.

**Tag Browser (bottom-left-ish).** Your tag tree. Drag a tag onto the canvas and Ignition offers to create a bound component. Drag a tag onto a *property* in the Property Editor to create a binding. That drag-to-bind gesture is the fastest way to work.

**Output Console** (`Tools > Output Console` if not visible). Where `print` from your Gateway-scoped scripts shows up while you're in the Designer, and where script errors surface. Keep it open.

## 2.3 Saving and the safety net

- `Ctrl+S` saves the project to the Gateway. Open sessions update.
- `Ctrl+Z` / `Ctrl+Y` undo/redo.
- **Project versioning:** 8.3's JSON-on-disk resource storage is what makes real source control practical. If your team has Git wired to the Gateway's project directory, your saves become reviewable commits. If not, `File > Export` a view before you make a risky change. Do this. It costs ten seconds and has saved me entire afternoons.

⚠ There is no "test environment" unless someone built one. On a dev Gateway you're safe; on production, **assume every save is live to operators.**

# Part 3. Views, Containers, and Layout

## 3.1 The View is the atom

A **View** is a self-contained, reusable chunk of UI. It has:

- a **root container** (chosen at creation — you can't easily change it later, so choose deliberately),
- **params** — its inputs and outputs (like function arguments),
- **props** — view-level settings such as `defaultSize` and `loading` behavior,
- **custom** — view-scoped variables.

Views are used three ways:

1. As a **page's primary view** (the main content of a URL).
2. As a **docked view** (nav bar, header, alarm banner).
3. **Embedded inside another view** (via the Embedded View component, Flex Repeater, or View Canvas).

 **Design every view as if someone else will embed it.** Take a parameter, don't reach out to globals, don't hardcode a tag path. That discipline is what turns a screen into a component library.

**Naming and folders.** Set your convention on day one and never break it:

```
Views/  
  Pages/           ← full screens that a URL points at  
    Overview  
    Line1/Detail  
  Docks/           ← nav bars, headers, alarm banners  
    LeftNav  
    Header  
  Popups/          ← modal/popup content  
    MotorFaceplate  
    ConfirmDialog  
  Components/      ← small reusable pieces  
    ValueCard  
    StatusPill
```

## 3.2 Containers: the five layout engines

The container type decides what the **POSITION** tab means for its children. This is the concept new developers fight hardest and it's genuinely simple once stated:

| Container  | Children positioned by                                                                                  | Use it when                                                                |
|------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Coordinate | <code>x</code> , <code>y</code> , <code>width</code> , <code>height</code> (fixed px or percent mode)   | P&ID-style mimics, overlapping/z-stacked graphics, anything drawn to scale |
| Flex       | <code>basis</code> , <code>grow</code> , <code>shrink</code> , <code>alignSelf</code> along a direction | <b>Your default.</b> Rows and columns that adapt to screen size            |

| Container         | Children positioned by                                               | Use it when                                  |
|-------------------|----------------------------------------------------------------------|----------------------------------------------|
| <b>Column</b>     | Responsive grid columns that reflow by screen size                   | Dashboard-style card grids                   |
| <b>Breakpoint</b> | Two children: one "large", one "small", swapped at a pixel threshold | Genuinely different desktop vs phone layouts |
| <b>Tab</b>        | Children become tabs                                                 | Tabbed sections inside one view              |

**Coordinate container** has two modes:

- **Fixed** — children keep their pixel size; the container may letterbox or clip.
- **Percent** — children scale proportionally with the container. This is how you make a P&ID diagram that grows with the window without rewriting it.

**Flex container** — the workhorse. Set `direction` to `row` or `column`, then for each child:

- `basis` — the starting size along the direction ( `auto` , `100px` , `30%` )
- `grow` — share of *leftover* space this child absorbs (0 = don't grow)
- `shrink` — willingness to give up space when cramped
- `alignSelf` — cross-axis alignment
- Container-level: `justify` (along the axis), `alignItems` (across it), `wrap`

If you know CSS flexbox, it is CSS flexbox. If you don't: `grow: 1` on the middle child and `grow: 0` on the header and footer gives you the classic header / body / footer layout, and it's correct at every screen size. That single trick covers most screens.

⚠ **The number one layout mistake:** building everything in a Coordinate container because it feels like drawing, then discovering the screen is unusable on a tablet. Default to **Flex**. Reach for Coordinate only when you're drawing a picture of a physical process.

### 3.3 View parameters: input and output

On the View's `PARAMS` tab you define parameters, each marked:

- **input** — passed in by whoever embeds/navigates to this view,
- **output** — the view writes back to the parent,
- **in/out** — both.

A faceplate view might take `tagPath` (input) and return `confirmed` (output).

Pass params when you:

- embed a view ( `props.params` on the Embedded View component),
- open a popup ( `system.perspective.openPopup(..., params={...})` ),
- navigate to a page whose URL has parameters ( `/line/:lineId` ).

### 3.4 Embedding and repetition

| Component     | What it does                                                       | Use when                            |
|---------------|--------------------------------------------------------------------|-------------------------------------|
| Embedded View | Renders one view inside another, passing <code>params</code>       | Reuse a faceplate, a card, a header |
| Flex Repeater | Renders <i>N</i> copies of one view from an array of param objects | A list of pumps, a set of KPI cards |
| View Canvas   | Renders multiple views at explicit positions                       | Dashboard-ish free placement        |

**Flex Repeater is the one to learn early.** Its `props.instances` is an array where each element is a dict of params for one copy:

```
[
  { "tagPath": "[default]Line1/Pump1", "label": "Pump 1" },
  { "tagPath": "[default]Line1/Pump2", "label": "Pump 2" },
  { "tagPath": "[default]Line1/Pump3", "label": "Pump 3" }
]
```

Bind `instances` to a query or a script and your UI grows and shrinks with your plant automatically. That's the moment Perspective stops being "drawing screens" and starts being software.

⚠ Repeater are not free. Each instance is a full view with its own bindings. 20 instances of a view with 15 bindings is 300 live subscriptions **per session**. Watch it.

## Part 4. Properties and Bindings

### 4.1 The property tree is the whole API

Everything in Perspective is a property on a JSON-ish tree. `props.text`, `props.style.backgroundColor`, `position.grow`, `meta.visible`, `custom.selectedMotor`. Bindings write to properties, scripts write to properties, the renderer reads properties. There is no hidden layer.

### 4.2 The binding types

Select a property, click the **binding icon** (chain link) next to it, and pick a type:

| Binding              | What it reads                                       | Notes                             |
|----------------------|-----------------------------------------------------|-----------------------------------|
| Tag                  | A tag's value (or any tag property)                 | Three flavors — see below         |
| Property             | Another property in this same view                  | The glue for view-local wiring    |
| Expression           | An Ignition expression                              | Cheap, fast, runs on value change |
| Expression Structure | Builds a JSON object/array from several expressions | Great for feeding charts/tables   |
| Query                | A <b>Named Query</b> or inline SQL                  | Polled or triggered               |
| Tag History          | Historical tag data as a dataset                    | Feeds charts and trend tables     |
| HTTP                 | A REST endpoint                                     | Pull external web data            |

**Tag binding, three flavors:**

1. **Direct** — you pick the tag in the browser. `[default]Line1/Tank1/Level`. Simple, static.
2. **Indirect** — the path contains placeholders filled from other properties:

```
[default]{1}/Level
```

where `{1}` is bound to `view.params.equipmentPath`.  **This is how one view serves 200 motors.** Indirect tag binding + a view parameter = the faceplate pattern. Learn it in Lab 8.

3. **Expression** — the whole tag path is computed by an expression. Most flexible, slightly harder to read.

**Binding options worth knowing:**

- **Bidirectional** (Tag and Property bindings) — writes flow *back*. A numeric input bound bidirectionally to a setpoint tag writes the setpoint when the operator types. ⚠ Use deliberately; an accidental bidirectional binding on a mimic is how an operator "changes" a value by looking at it.
- **Overlay Opt-Out** — suppress the bad-quality overlay for this binding. Use sparingly and only where you handle quality yourself.
- **Enabled** — turn a binding off without deleting it. Handy for debugging.

## 4.3 Transforms: the pressure valve

A binding can carry a chain of **transforms** that reshape the value on its way to the property:

| Transform         | Does                                                                           | Example                                                |
|-------------------|--------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>Map</b>        | Lookup table, input → output                                                   | <code>0 → "Stopped", 1 → "Running", 2 → "Fault"</code> |
| <b>Format</b>     | Number/date formatting                                                         | <code>#,##0.0</code> or <code>yyyy-MM-dd HH:mm</code>  |
| <b>Expression</b> | An expression where <code>value</code> is the incoming value                   | <code>value * 2.20462</code>                           |
| <b>Script</b>     | Python <code>def transform(self, value, quality, timestamp): return ...</code> | Anything else                                          |



**Transforms are how you avoid creating junk tags.** Need level shown as a percentage *and* as a bar color? One tag binding, two components, transforms doing the last mile. Don't make a `Level_Percent` memory tag for a display concern.



**Script transforms are the most abused feature in Perspective.** They run on the Gateway every time the value changes, for every session. A `system.db.runQuery` inside a script transform on a 1-second tag, open in 40 sessions, is 40 queries a second. Use Map and Expression first; reach for Script when you truly need it, and keep it pure and fast.

## 4.4 The expression language in 90 seconds

Not Python. A small, fast, side-effect-free expression language.

```
// tag reference
{[default]Line1/Tank1/Level}

// property reference in the same view
{view.params.lineId}
{../Label.props.text}           // relative path to a sibling component

// conditionals
if({[default]Pump1/Running}, "RUNNING", "STOPPED")

case({[default]Pump1/State}, 0, "Off", 1, "Starting", 2, "Running", "Unknown")

// math & strings
round({[default]Tank1/Level} / {[default]Tank1/Capacity} * 100, 1)
concat("Line ", {view.params.lineId}, " - ", toStr(now()))

// null safety – do this more than you think you need to
coalesce({[default]Maybe/Missing}, 0)

// dates
dateFormat(now(), "yyyy-MM-dd HH:mm:ss")
dateDiff({session.props.startTime}, now(), "minute")
```

⚠ `now()` in an expression takes a poll rate argument ( `now(1000)` ) and **forces the expression to re-evaluate at that rate**. `now(0)` evaluates once. A screen full of `now()` at default rate is a screen that never stops working. Be intentional.

## 4.5 Where to put logic: the decision table

New developers scatter logic everywhere. Use this order of preference:

| Need                                                                        | Put it in                                         | Why                                                |
|-----------------------------------------------------------------------------|---------------------------------------------------|----------------------------------------------------|
| Show a tag                                                                  | Tag binding                                       | Cheapest                                           |
| Reformat/recolor a value for display                                        | Transform on the binding                          | Stays with the display                             |
| Value derived from several tags, used by one view                           | Expression binding on a custom property           | Local, visible, cheap                              |
| Value derived from tags, needed by <b>many</b> screens or by alarms/history | <b>Expression tag</b>                             | Computed once on the Gateway, not once per session |
| Data from SQL                                                               | <b>Named Query</b> binding                        | Parameterized, cached, secured, reusable           |
| Reusable Python                                                             | <b>Project Library</b> script, called from events | Testable, one copy                                 |
| Something that must run without a session open                              | <b>Gateway Event</b> (timer/tag change)           | Sessions come and go                               |

🔑 The question that settles most design arguments: *"Does this need to be true when nobody is looking at it?"* If yes → tag or gateway event. If no → binding in the view.

## 4.6 The view model pattern

The pattern that separates maintainable Perspective from spaghetti:

1. On the view's **root container**, create `custom` properties that describe the view's state — `custom.motorData`, `custom.selectedIndex`, `custom.isEditing`.
2. Bind those custom properties to tags / queries / expressions. **This is the only place data enters the view.**
3. Bind each component's display properties to those custom properties with simple **Property bindings**.
4. Component events write to custom properties, not to other components.

Why: one place to look when data is wrong, one place to change when the source changes, and components become swappable. Contrast with the trap — a Label bound to a tag, a Gauge bound to the same tag, a script that reaches into `../Flex/Label.props.text` to read it back. That project becomes unmaintainable at about screen fifteen, reliably.

# Part 5. Events, Actions, and Scripting

## 5.1 Events and configured actions

Select a component, open the **Event Configuration** dialog (the ⚡ / gear icon on the component, or right-click). You'll see events grouped by kind:

- **Mouse:** `onClick`, `onDoubleClick`, `onMouseEnter`, `onMouseLeave`, `onContextMenu`
- **Component-specific:** `onActionPerformed` (Button), `onChange` (inputs), `onRowClick` (Table)
- **Keyboard, Focus**, and others depending on the component

For each event, attach one or more **actions**. You rarely need a script:

| Action        | Does                                         |
|---------------|----------------------------------------------|
| Navigation    | Go to a page, a view, or a URL               |
| Popup         | Open / close / toggle a popup, with params   |
| Dock          | Open / close / toggle a docked view          |
| Set Property  | Write a value to a property — no code needed |
| Script        | Run Python                                   |
| Download File | Send a file to the browser                   |

🔑 Prefer configured actions over scripts. They're visible in the Designer, they show up in search, and they don't hide behavior from the next person.

## 5.2 Scripting: what you must know

Ignition scripting is **Jython 2.7** — Python 2.7 syntax on the JVM. Practical consequences:

- `print x` and `print(x)` both work; use `print(...)`.
- Integer division: `1/2` is `0`. Write `1.0/2`.
- You have access to Java classes if you need them.
- Third-party CPython packages with C extensions (numpy, pandas) are **not** available. Pure-Python libraries can be added, but plan your logic around SQL and Ignition's own functions.

**Inside a Perspective event script you get:**

```
def runAction(self, event):
    # self -> the component the script is attached to
    # self.view -> the view
    # self.view.params.tagPath -> a view parameter
    # self.getSibling("Label") -> a sibling component
    # self.getChild("Flex").getChild("Label")
    # self.parent -> the parent component
    # self.session -> session object (props, user info)
    # self.page -> page object
    pass
```

### Writing properties from a script:

```
def runAction(self, event):
    self.getSibling("StatusLabel").props.text = "Working..."
    self.view.custom.busy = True
```

### Reading and writing tags:

```
def runAction(self, event):
    # read
    qv = system.tag.readBlocking(["[default]Line1/Pump1/Speed"])[0]
    speed = qv.value

    # write
    system.tag.writeBlocking(
        ["[default]Line1/Pump1/SpeedSetpoint"],
        [1750]
    )
```

⚠ `readBlocking` / `writeBlocking` block the Gateway thread. Never loop over 500 tags one at a time — pass the whole list in one call, which is exactly what those array arguments are for.

## 5.3 The Project Library

Project Browser > Scripting > Project Library. Create a package (say `plant`) and a module (`motors`):

```
# Project Library > plant > motors
def start(tagPath):
    """Command a motor to start. Returns True if the write was accepted."""
    result = system.tag.writeBlocking([tagPath + "/StartCmd"], [True])
    return result[0].good

def summarize(tagPaths):
    """Read Running/Fault for many motors in ONE call."""
    paths = []
    for p in tagPaths:
        paths.append(p + "/Running")
        paths.append(p + "/Fault")
    values = system.tag.readBlocking(paths)
    out = []
    for i, p in enumerate(tagPaths):
```

```

        out.append({
            "path": p,
            "running": values[i * 2].value,
            "fault": values[i * 2 + 1].value,
        })
    return out

```

Call it from anywhere:

```

def runAction(self, event):
    plant.motors.start(self.view.params.tagPath)

```

 Rule I enforce on every team: **event scripts are three lines or fewer**. Validate, call the library, handle the result. All real logic lives in the Project Library where it's reusable and where you can test it in the Script Console.

**Script Console** (Tools > Script Console) runs Python against the Gateway right now. It is your REPL. Develop every non-trivial script there first.

## 5.4 Message handlers: components talking without touching

You need the nav bar to tell the main view "refresh". They're in different views and can't reach each other. Use messages.

**Sender** (a script action):

```

def runAction(self, event):
    system.perspective.sendMessage(
        "refreshData",
        payload={"lineId": 1},
        scope="page"           # "page", "session", or "view"
    )

```

**Receiver** — on the target component or view: Event Configuration > **Message Handlers** > add `refreshData`, matching scope:

```

def onMessageReceived(self, payload):
    lineId = payload["lineId"]
    self.view.custom.lineId = lineId
    self.refreshBinding("custom.data")

```

Scopes:

- **view** — only handlers in the same view instance
- **page** — every view on the current page, including docks and popups
- **session** — every page in the session (the whole browser tab set for that user)

 Messages are the decoupling tool. A popup that saves a record shouldn't know who opened it — it broadcasts "recordSaved" and whoever cares reacts.

## 5.5 Gateway and session events

Project Browser > Scripting:

- **Gateway Events** — run on the Gateway with **no session required**:
  - **Timer** — every N ms/seconds (fixed delay or fixed rate)
  - **Tag Change** — when configured tags change
  - **Startup / Shutdown**
  - **Message** — handlers for `system.util.sendMessage`
- **Session Events** — Perspective-specific, per session:
  - **Startup / Shutdown** — session begins/ends
  - **Page Startup**
  - **Authentication** — after a user logs in (great place to set `session.custom` defaults)

⚠ Gateway timer scripts are where performance goes to die. Every one you add runs forever on the production server. Before writing one, ask whether a tag change event or a transaction group would do the job with less risk.

## Part 6. Data: Tags, History, and SQL

---

### 6.1 Getting real data in

Gateway web UI > Config :

- **OPC UA** — Ignition includes an OPC UA server and drivers (Allen-Bradley, Siemens, Modbus TCP...). Create a **Device Connection**, browse it in the Designer's OPC Browser, drag addresses into your tag tree to create OPC tags.
- **Database Connections** — JDBC to SQL Server/Postgres/MySQL/etc. The connection has a name; your queries reference that name.
- **MQTT** — via Cirrus Link modules if installed.

For training, you do not need a PLC. **Memory tags plus expression tags simulate a plant fine**, and Ignition ships simulator devices (Programmable Device Simulator / generic simulators) you can add as a device connection to get moving values for free. Lab 2 uses memory tags; Lab 12 suggests the simulator.

### 6.2 Tag history

Enable history per tag (or per UDT definition member — do it there, once, for all instances):

- **Historical Scanclass / sample mode** — how often to evaluate storing
- **Deadband** — don't store unless it moved by X (analog compression; this is what keeps your database from exploding)
- **Max time between samples** — store periodically even if unchanged, so gaps aren't ambiguous

Then bind a chart to a **Tag History binding**: pick tag paths, a time range (or bind the range to date pickers), an aggregation mode ( `Average` , `MinMax` , `LastValue` ...), and return format.

**Perspective components for history:** `Power Chart` (operator-facing, users pick their own pens — a huge amount of free functionality), `Time Series Chart` (developer-controlled), `XY Chart` , `Simple Gauge` , tables.

**8.3 changed the historian substantially.** There is now a **Historian Core Module** plus a **SQL Historian Module** and a documented **Historian API**, with a fast built-in time-series store. From the Designer, binding to history feels the same; the difference is in Gateway configuration and performance characteristics. When you set up history on your dev Gateway, look at `Config > Historian` in the new Gateway UI and note which provider you're writing to.

### 6.3 Named Queries: the only way you should hit SQL

`Project Browser > Named Queries` . A Named Query is a stored, parameterized, permission-checked, optionally cached SQL statement.

Types: **Query** (returns a dataset), **Update Query** (INSERT/UPDATE/DELETE, returns rows affected), **Scalar Query** (returns one value).

```
-- Named Query: Production/GetShiftTotals
SELECT line_id, SUM(units) AS total
FROM production_log
WHERE ts >= :startDate AND ts < :endDate
GROUP BY line_id
ORDER BY line_id
```

Parameters are typed on the Authoring tab ( `startDate` : DateTime, `endDate` : DateTime). Bind a component's `props.data` with a **Query binding** pointing at this Named Query, and bind each parameter to a date picker's value.

**Why Named Queries and not `system.db.runQuery` with a string:**

|                  | Named Query                  | Inline string SQL                       |
|------------------|------------------------------|-----------------------------------------|
| SQL injection    | Parameters are bound — safe  | You are one concatenation from a breach |
| Caching          | Built in                     | None                                    |
| Security         | Per-query permissions        | None                                    |
| Reuse            | One definition, many callers | Copy-paste                              |
| Finding it later | In the tree                  | Buried in a script somewhere            |

⚠ **Value vs Literal parameters.** `Value` parameters are properly bound (safe). `Literal` parameters are string-substituted into the SQL — needed for dynamic table/column names, and **exactly** as dangerous as it sounds. Never wire a Literal parameter to anything a user can type.

Call one from script when you need to:

```
params = {"startDate": start, "endDate": end}
data = system.db.runNamedQuery("Production/GetShiftTotals", params)
```

## 6.4 Datasets vs arrays

Two shapes of tabular data, and mixing them up burns a whole afternoon:

- **Dataset** — Ignition's native table type. What SQL queries and tag history return.
- **JSON array of objects** — what Perspective components generally want in `props.data`.

Perspective's Query binding usually offers a return format that gives you what the component needs. When you need to convert in script:

```
# dataset -> list of dicts
data = system.dataset.toPyDataSet(ds)
rows = []
for row in data:
    rows.append({"line": row["line_id"], "total": row["total"]})
```

If a Table shows nothing and threw no error, check this first. It is almost always this.

# Part 7. Pages, Docks, Popups, and Navigation

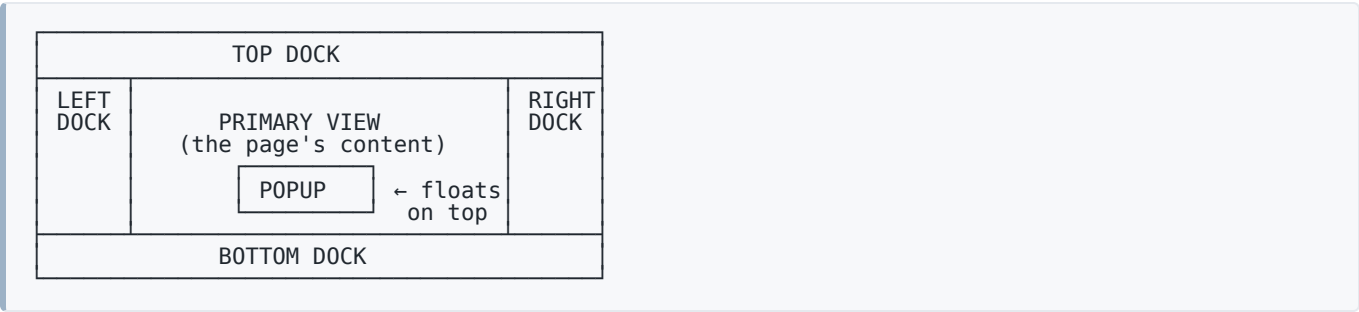
## 7.1 Page Configuration

Project Browser > Perspective > Page Configuration . Here you map **URL paths** → **views**:

| URL           | Primary View     |
|---------------|------------------|
| /             | Pages/Overview   |
| /line/:lineId | Pages/LineDetail |
| /reports      | Pages/Reports    |

A `:parameter` segment in the URL becomes a view parameter. `/line/3` passes `lineId = "3"` into the view. That means your screens are **linkable** — an operator can bookmark a specific machine, and you can email someone a URL that opens exactly the screen you mean. Use it.

## 7.2 The regions of a page



**Docked views** are configured in Page Configuration. Each dock gets an **ID** (name it — you'll need it for `system.perspective.toggleDock("leftNav")` ). Key settings:

- **Display** — `visible` (always there), `onDemand` (opens when asked), `auto` (breakpoint-driven)
- **Anchor / Push vs Overlay** — does the dock push the content aside or float over it
- **Size, resizable, modal, auto-dismiss**
- **Show when** — breakpoint conditions, so the nav is a permanent sidebar on desktop and a hamburger drawer on a phone

**Shared vs per-page docks.** You can configure docks on the "Shared Settings" so every page gets the nav bar, then override for specific pages (a full-screen kiosk view with no nav). Set your nav up once in Shared Settings — do not paste it into every page.

## 7.3 Popups

```
system.perspective.openPopup(  
    id="motorFaceplate",           # unique handle; reopening same id reuses it  
    view="Popups/MotorFaceplate",  
    params={"tagPath": "[default]Line1/Pump1"},  
    title="Pump 1",  
    modal=True,  
    draggable=True,  
    resizable=False,  
    showCloseIcon=True  
)  
  
system.perspective.closePopup("motorFaceplate")
```

△ The `id` is how you control it later. Give every popup a deliberate id. If you open popups with generated ids you cannot close them programmatically and they stack up on the operator.

The **Popup action** on an event does all of this without code — prefer it.

## 7.4 Navigation

```
system.perspective.navigate(page="/line/3")           # internal page  
system.perspective.navigate(url="https://example.com") # external  
system.perspective.navigate(view="Pages/Detail", params={...}) # swap the primary view
```

🔑 Prefer `page=` navigation over `view=`. Page navigation updates the URL, works with the browser back button, and is bookmarkable. View navigation does not and confuses operators who expect the back button to work.

## 7.5 Session and page properties

```
session.props.auth.user.userName    # who is logged in  
session.props.auth.user.roles       # their roles  
session.props.auth.authenticated  
session.props.device.type           # desktop / mobile / tablet  
session.props.address               # client IP  
session.props.theme                 # active theme name  
session.props.locale  
  
session.custom.*                    # YOUR session-scoped variables  
  
page.props.pageId                   # unique id of this page instance
```

`session.custom` is the right place for "the plant this operator selected" or "their preferred units" — set it in a Session Startup or Authentication event, read it anywhere. It is per-session, so two operators can be looking at different lines in the same project simultaneously.

## Part 8. Styling and Themes

### 8.1 Three levels, use them in this order

1. **Inline style** — `props.style.backgroundColor` on one component. Fine for one-offs, terrible as a habit.
2. **Style Classes** — `Project Browser > Perspective > Styles`. Define `status.running`, `status.fault`, `card.base` once; apply via `props.style.classes`. **This is where your styling should live.**
3. **Themes** — Gateway-level CSS that defines the palette ( `light`, `dark`, and custom themes). Switch at runtime with `system.perspective.setTheme("dark")`.

🔑 `props.style.classes` accepts a **space-separated list** and can be **bound**. This is the whole trick for state-driven styling:

```
// expression binding on props.style.classes
"card.base " + case({[default]Pump1/State},
  0, "status.stopped",
  1, "status.starting",
  2, "status.running",
  "status.unknown")
```

One binding, no scripts, colors defined in exactly one place. When the plant standard changes from green to teal, you edit one style class rather than 400 components.

### 8.2 Style class discipline

Name by *meaning*, not appearance: `status.fault`, not `text.red`. In two years the fault color will change and `text.red` rendering blue is a special kind of misery.

Build a tiny style system before your first real screen:

```
Styles/
status/   running, stopped, fault, warning, disabled
text/     title, subtitle, body, caption, value.large
card/     base, header, body
layout/   page.padding, section.gap
```

### 8.3 Symbols and the 8.3 Drawing Editor

Perspective ships a **Symbol** library (motors, valves, pumps, vessels) with built-in state properties — bind `props.value` to a state tag and the symbol animates. Use these before drawing anything yourself.

**New in 8.3: the Drawing Editor.** A real vector illustration tool built into the Designer — paths, fills and strokes, snapping, guides, layering, and animation of what you draw, without leaving Ignition and without a round trip through Inkscape/Illustrator and SVG imports. If you need a custom P&ID element or a plant-specific symbol, this is now the native way to make it. Worth an hour of play once you're through the labs — it's a side quest, not a prerequisite.

# Part 9. Security

---

## 9.1 The pieces

- **Identity Providers (IdP)** — where users come from. Ignition's internal user source, or LDAP/AD, or OIDC/SAML SSO. Configured in the Gateway, assigned per project.
- **Users, Roles** — the classic model. A user has roles; you check roles.
- **Security Levels** — 8.x's hierarchical model, a tree like `Authenticated/Roles/Operator`, `Authenticated/Roles/Supervisor`. More expressive than flat roles.
- **Security Zones** — *where* the request comes from (IP ranges, host). "Setpoint changes only from the control-room subnet" is a security zone rule.
- **Permissions** — combine levels and zones: *this action requires Supervisor **and** ControlRoom zone.*

## 9.2 Applying it in Perspective

**On components** — Component permissions let you set what happens when the user lacks permission: hide the component, or disable it. Set this on the Button that starts the pump, not just on the page.

**In expressions / scripts:**

```
if system.perspective.isAuthorized(  
    isAllOf=False,  
    securityLevels=[{"name": "Authenticated/Roles/Supervisor"}]):  
    ...
```

```
// expression binding on meta.visible  
isAuthorized(false, "Authenticated/Roles/Supervisor")
```

**On Named Queries** — per-query permissions, so the data itself is protected, not just the button.

 **Client-side hiding is not security.** Hiding a button stops a confused operator, not a determined one. The real enforcement belongs on the Named Query, on the tag's write permissions, and in the security zone. Do both: hide *and* enforce.

## 9.3 Audit

Turn on an **Audit Profile** in the Gateway and point your project at it. Tag writes, query executions, and logins get logged with who/what/when. On any system that touches production you want this on from day one — the first time someone asks "who changed the setpoint at 2am," you will either have an answer or an awkward meeting.

# Part 10. Alarms

---

## 10.1 Configuring

Alarms are configured **on the tag**, not on the screen. Select a tag, open the Alarms editor, add an alarm with:

- **Name, Priority** (Diagnostic → Critical)
- **Mode**: Above Setpoint, Below Setpoint, Between, Equal, Bit State, On Change...
- **Setpoint, Deadband, Time On/Off Delay** (delays are how you stop chattering alarms)
- **Display Path** — the operator-friendly name. Set it. Raw tag paths in an alarm table are hostile.
- **Associated Data** — extra context captured at alarm time (batch number, operator, line speed). Enormously useful in post-incident review and almost always skipped.

Configure alarms on the **UDT definition** and every instance inherits them. Override per instance where needed.

## 10.2 Displaying

- **Alarm Status Table** — live alarms; ack and shelve from it
- **Alarm Journal Table** — history of alarm events, queried from the journal database

Both filter by source path, priority, state, and display path. A common pattern: an always-visible bottom dock with a compact alarm banner, bound to a filtered Alarm Status Table.

## 10.3 Notification

The Alarm Notification module plus pipelines (email, SMS, voice) handles escalation and on-call rosters. Out of scope for your first week, but know that "alarm happened" and "someone got told" are two separate systems.

## Part 11. What Is New in 8.3

---

You're learning on 8.3.9, so you get these for free — but most tutorials and forum answers online were written for 8.1. When something looks different from a video you're watching, this list is usually why.

### Perspective

- **Drawing Editor** — native vector illustration inside the Designer: paths, fills/strokes, snapping, guides, layering, and animation. Custom graphics without an external SVG tool.
- **Form generator** — configure sections, fields, conditional logic and validation rules once; the component builds the responsive layout, internal data logic, and client-side validation.
- **Offline mode** — the Perspective mobile app can accept form entry with no connectivity and sync when the link returns. Genuinely new capability for remote-site data collection.

### Gateway

- **Redesigned web interface** — new navigation, global search, and a cleaner connector/config experience. If you follow an older tutorial's "go to Configure > ..." path and it doesn't exist, search for the page name instead.

### Historian

- A restructured historian: **Historian Core Module**, **SQL Historian Module**, and a public **Historian API** for custom backends, with a fast built-in time-series store. Query and binding ergonomics in the Designer are familiar; Gateway-side configuration is the part that changed.

### Event Streams (new resource type)

- A structured pipeline: a **source** (tag change, Gateway event, HTTP, Kafka, database), through **encoding / filtering / transformation** stages, into a **handler** (script, database table, etc.), with a dedicated **error-handling stage** and a **test mode**. This is the clean answer to a whole category of problems people used to solve with a pile of Gateway tag-change scripts. Worth knowing it exists on day one even if you won't use it in week one.

### Project storage and DevOps

- Project resources are stored as **JSON files on disk**, making change tracking, merging, and Git workflows practical. This is the big structural change of 8.3 and the reason a lot of shops upgraded.

△ Point releases (8.3.1 → 8.3.9) add fixes and refinements. When a behavior surprises you, check the release notes for versions between the tutorial you're following and yours.

# Part 12. The Training Plan

---

Seven working sessions of 2–3 hours, then a capstone. Each session: read, build, then answer the check questions without looking. **Do not skip the check questions** — they're how you find out you've been pattern-matching instead of understanding.

If you only have a few hours total, do Days 1, 2, and 4. Those three cover 70% of daily Perspective work.

---

## Day 1 — Orientation and your first live value (2–3 h)

**Read:** Parts 1 and 2.

**Build:**

- [Lab 1: Get Oriented](#) — Gateway UI, Designer launch, create a project
- [Lab 2: Make Some Tags](#) — memory + expression tags simulating a tank
- [Lab 3: Hello, Tag](#) — first view, first binding, first session

**You are done when:** a number changes in your browser because you changed a tag in the Designer, and you can explain out loud which machine did the computing.

**Check questions:**

1. Where does a Perspective script execute?
  2. What are the three things the Gateway does for a running session?
  3. What's the difference between the Designer and a session?
- 

## Day 2 — Layout that survives a tablet (2–3 h)

**Read:** Part 3.

**Build:**

- [Lab 4: Flex Layout](#)
- [Lab 5: Coordinate and Percent Mode](#)
- [Lab 6: A Real Page with Docks](#)

**You are done when:** you can resize the browser from desktop width to phone width and your screen stays usable without a horizontal scrollbar.

**Check questions:**

1. What does `position.grow` do, and on which container types does it exist?
  2. When would you genuinely choose Coordinate over Flex?
  3. Where are docked views configured, and how do you open one from a script?
-

## Day 3 — Bindings, transforms, expressions (2–3 h)

**Read:** Part 4.

**Build:**

- [Lab 7: Five Bindings](#)
- [Lab 8: The Faceplate Pattern](#) ← **the most important lab in this guide**

**You are done when:** you have one view that displays any motor in your plant based on a parameter, and you've dropped three copies of it on a screen pointed at three different motors.

**Check questions:**

1. Name four binding types and when you'd pick each.
  2. What is an indirect tag binding and what problem does it solve?
  3. Where would you put logic that must be true even with zero sessions open?
- 

## Day 4 — Reuse: parameters, embedding, repeaters (2–3 h)

**Read:** Parts 3.3–3.4, re-read 4.6.

**Build:**

- [Lab 9: Flex Repeater](#)
- [Lab 10: UDTs](#)

**You are done when:** adding a new pump to your tag tree makes a new card appear on your screen **without you touching the view**.

**Check questions:**

1. What shape does `props.instances` on a Flex Repeater expect?
  2. If you change a UDT definition, what happens to instances?
  3. Why does a view taking a `tagPath` parameter beat a view with a hardcoded path?
- 

## Day 5 — SQL, history, and charts (2–3 h)

**Read:** Part 6.

**Build:**

- [Lab 11: Named Query to Table](#)
- [Lab 12: History and a Power Chart](#)

**You are done when:** a table on screen updates when you change a date picker, and you have a trend chart showing a tag's actual recorded history.

**Check questions:**

1. Why a Named Query instead of inline SQL? Give three reasons.
  2. What's the difference between a Value and a Literal query parameter?
  3. What does a history deadband do and why do you care?
- 

## Day 6 — Scripting, events, messages (2–3 h)

**Read:** Part 5.

**Build:**

- [Lab 13: Buttons That Do Things](#)
- [Lab 14: Popups with Parameters](#)
- [Lab 15: Message Handlers](#)

**You are done when:** a button in a docked nav bar causes a view in the page center — which it has no direct reference to — to refresh.

**Check questions:**

1. What are the three message scopes and when do you use each?
  2. Why should event scripts be short?
  3. What is `self` inside a Perspective event script?
- 

## Day 7 — Alarms, security, style, and performance (2–3 h)

**Read:** Parts 8, 9, 10, 15.

**Build:**

- [Lab 16: Alarms End to End](#)
- [Lab 17: Style Classes and Theming](#)
- [Lab 18: Lock It Down](#)

**You are done when:** an alarm you configured shows up in an alarm table, your colors come from style classes rather than inline styles, and a button is hidden *and* enforced for unauthorized users.

**Check questions:**

1. Why is hiding a button not security?
  2. Where are alarms configured, and why on the UDT definition?
  3. What's the difference between a style class and a theme?
- 

## Capstone — Line Monitoring Application (one full session, 4+ h)

Build, from scratch, without following steps:

**Requirements**

1. A UDT called **Motor** with **Running**, **Fault**, **Speed**, **SpeedSetpoint**, **RunHours**. History on **Speed** and **RunHours**. A high-speed alarm and a fault alarm on the definition.
2. At least six instances across two lines.
3. A left nav dock (line selection) and a top header dock (plant name, clock, logged-in user, alarm count badge).
4. An **Overview** page: a Flex Repeater of motor status cards, driven by a query or a tag-browse script, not a hardcoded list.
5. Clicking a card opens a **faceplate popup** for that motor: live values, a speed setpoint the operator can write, a start/stop button, a 1-hour trend.
6. The start/stop button is **hidden and enforced** for non-Supervisors.
7. All colors from style classes. Works in light and dark theme.
8. A **Reports** page with a date range and a table from a Named Query.
9. Usable at 1920px and at 400px wide.

### Self-review before you call it done

- Is there a single hardcoded tag path in any reusable view? (There shouldn't be.)
- Does any script exceed ten lines outside the Project Library?
- Does any component reach into another component's properties by relative path?
- How many bindings are on your heaviest screen? Could any move to an expression tag?
- Open it on your phone. Honestly — is it usable?

That capstone is roughly the shape of a real first assignment. Build it and you're employable on Perspective.

# Part 13. Labs

---

Every lab: **Goal** → **Steps** → **Done when** → **Common mistakes**.

---

## Lab 1: Get Oriented

**Goal:** Know where things live.

### Steps

1. Browse to your Gateway ( `http://<host>:8088` ). Log in.
2. Find and note: **Status** (health, sessions, performance), **Config** (connections, security, historian), and the project list. In 8.3's redesigned UI, use the **global search** box — it's faster than hunting menus.
3. `Status > Systems > Perspective Sessions` — you'll come back here constantly to see who's connected.
4. Download and install the **Designer Launcher**, add your Gateway, launch the Designer.
5. Create a new project: name it `training`, no parent project, set it as a Perspective project.
6. In the Designer, open `Tools > Script Console` and run:

```
print(system.date.now())
print(system.tag.readBlocking(["[System]Gateway/SystemName"])[0].value)
```

**Done when:** the Script Console prints your Gateway's name. You have just executed code on the server from your desk — that's the whole architecture in one line.

**Common mistakes:** installing the *Vision Client Launcher* instead of the *Designer Launcher*; using `localhost` from a different machine than the Gateway.

---

## Lab 2: Make Some Tags

**Goal:** A small simulated tank so you have moving data without a PLC.

### Steps

1. In the Designer's **Tag Browser**, right-click `Tags` (the `default` provider) > `New Tag > Folder`. Name it `Line1`, then a subfolder `Tank1`.
2. Inside `Tank1`, create these:

| Name                     | Type   | Data Type | Value / Expression |
|--------------------------|--------|-----------|--------------------|
| <code>Capacity</code>    | Memory | Float     | <code>1000</code>  |
| <code>Level</code>       | Memory | Float     | <code>450</code>   |
| <code>Setpoint</code>    | Memory | Float     | <code>500</code>   |
| <code>PumpRunning</code> | Memory | Boolean   | <code>false</code> |

| Name         | Type       | Data Type | Value / Expression                                                              |
|--------------|------------|-----------|---------------------------------------------------------------------------------|
| LevelPercent | Expression | Float     | {[.]Level} / {[.]Capacity} * 100                                                |
| Status       | Expression | String    | if({[.]LevelPercent} > 90, "HIGH", if({[.]LevelPercent} < 10, "LOW", "NORMAL")) |

Note `{[.]Level}` — the `[.]` means "relative to this tag's folder". Use it; it makes folders copy-pasteable.

3. Change `Level` to `950` in the Tag Browser and watch `LevelPercent` and `Status` recompute immediately. That is the Gateway's tag engine working.

**Done when:** editing `Level` changes two other tags with no code anywhere.

**Common mistakes:** creating the expression tag with datatype Integer (you'll get truncated percentages); typing the expression into the *value* field instead of the **Expression** field.

## Lab 3: Hello, Tag

**Goal:** First view, first binding, first session.

### Steps

1. **Project Browser > Perspective > Views** > right-click > **New View**.
  - Name: `Pages/Overview` (creating the folder inline is fine)
  - Root container type: **Flex**
  - Check "Set as the primary view for a new page" if offered, and give it page URL `/`
2. Drag a **Label** onto the canvas from the Component Palette.
3. Select it. In the Property Editor, find `props.text`. Click the **chain-link binding icon**.
4. Choose **Tag**, browse to `[default]Line1/Tank1/LevelPercent`, OK.
5. Add a **Format** transform: `#,###0.0` — click "Add Transform" in the binding dialog.
6. Drag a **Cylindrical Tank** (or Simple Gauge) from the palette. Bind its `props.value` to `[default]Line1/Tank1/LevelPercent`.
7. Drag a **Numeric Entry Field**. Bind `props.value` to `[default]Line1/Tank1/Setpoint`, and in the binding dialog check **Bidirectional**.
8. `Ctrl+S` to save.
9. Toggle **Preview Mode** in the toolbar. Type a new setpoint. Check the Tag Browser — the tag changed.
10. Now launch the real thing: from the Gateway home page, open your project's Perspective session (or go to `http://<host>:8088/data/perspective/client/training`). Put it side by side with the Designer. Change `Level` in the Tag Browser and watch the browser update.

**Done when:** two windows — Designer and browser — stay in sync with no refresh.

**Common mistakes:** forgetting to save (sessions show the last saved state); binding `props.text` on the tank instead of `props.value`; expecting the bidirectional write to happen in design mode (it needs Preview mode or a real session).

## Lab 4: Flex Layout

**Goal:** Build the header/body/footer layout that underlies most screens.

### Steps

1. New view `Sandbox/FlexPractice`, root container **Flex**.
2. Select the root. Set `props.direction = column`.
3. Drop three **Flex Containers** inside it. Rename them (in META > name) `Header`, `Body`, `Footer`.
4. Select `Header`: `position.basis = 60px`, `position.grow = 0`, `position.shrink = 0`.
5. Select `Footer`: same, `basis = 40px`, `grow = 0`, `shrink = 0`.
6. Select `Body`: `position.grow = 1`, `position.basis = auto`.
7. Give each a different `props.style.backgroundColor` temporarily so you can see them.
8. Select `Body`, set `props.direction = row`. Drop three Labels in it. Set each Label's `position.grow = 1`. They split the width evenly.
9. Change one Label's `grow` to `2`. It now takes half. That's the entire flex model.
10. Preview and resize the Designer's canvas width. Header and footer stay put; body absorbs.

**Done when:** you can predict what happens before you change a `grow` value.

**Common mistakes:** setting `grow` on the root container instead of its children; leaving `basis` at a fixed pixel value and wondering why `grow` seems ignored (it only distributes *leftover* space).

---

## Lab 5: Coordinate and Percent Mode

**Goal:** Know when the other tool is the right one.

### Steps

1. New view `Sandbox/CoordPractice`, root container **Coordinate**.
2. On the root, note `props.mode`. Leave it `fixed` for now.
3. Drop a **Rectangle** (Shapes/Drawing category) sized to look like a tank, and a **Cylindrical Tank** on top of it, overlapping. Note that overlap is trivial here and impossible in Flex — that's the point.
4. Add a Label positioned over the tank showing `[default]Line1/Tank1/Status`.
5. Preview and resize. Everything stays pixel-fixed.
6. Now set root `props.mode = percent`. Resize again — everything scales proportionally.
7. Compare: which behavior would you want for a P&ID mimic on a 4K control-room display versus a laptop? (Percent.) For a form? (Neither — use Flex.)

**Done when:** you can state the rule: *Coordinate for pictures of physical things, Flex for everything else.*

---

## Lab 6: A Real Page with Docks

**Goal:** A shell you'll reuse forever.

## Steps

1. Create `Views/Docks/LeftNav`, root **Flex**, `direction = column`. Add three **Buttons**: "Overview", "Line 1", "Reports".
2. Create `Views/Docks/Header`, root **Flex**, `direction = row`. Add:
  - a Label with the plant name (`position.grow = 0`)
  - a spacer Flex container with `position.grow = 1`
  - a Label bound by expression to `now(1000)` with a Format transform `HH:mm:ss`
  - a Label bound by expression to `{session.props.auth.user.userName}`
3. Open **Perspective > Page Configuration**.
4. Select **Shared Settings** (so these docks apply to all pages). Add a **Left** dock:
  - View: `Docks/LeftNav`, **ID**: `leftNav`, **Size**: `220`, **Display**: `visible`
5. Add a **Top** dock: View: `Docks/Header`, **ID**: `header`, **Size**: `60`, **Display**: `visible`.
6. Make sure `/` maps to `Pages/Overview`. Add a page `/reports` → create a stub view for it.
7. On the "Overview" button in LeftNav: Event Configuration > `onActionPerformed` > **Navigation** action > Page > `/`. Do the same for the others.
8. Save, open a session. Click around. Watch the URL change. Press the browser **back** button — it works, because you used page navigation.

**Done when:** you have a persistent nav and header on every page, configured in exactly one place.

**Common mistakes:** configuring docks on a single page instead of Shared Settings, then copying them to every page; forgetting to give the dock an ID (you need it for `toggleDock`).

---

## Lab 7: Five Bindings

**Goal:** Fluency with the binding dialog.

Build one view, `Sandbox/BindingZoo`, with five Labels:

1. **Tag binding** — `props.text` → `[default]Line1/Tank1/Level`.
2. **Expression binding** —

```
"Tank is " + toStr(round({[default]Line1/Tank1/LevelPercent}, 1)) + "% full"
```

3. **Property binding** — add a custom property on the root: select root > CUSTOM tab > add `custom.tankName` = "Tank 1". Bind Label 3's `props.text` to `view.custom.tankName` via a **Property** binding. Change the custom property's value and watch the label follow.
4. **Tag binding + Map transform** — bind `props.text` to `[default]Line1/Tank1/PumpRunning`, add a **Map** transform: `true` → "Pump Running", `false` → "Pump Stopped".
5. **Expression binding on `props.style.classes`** (after Lab 17, or use inline color now):

```
case({[default]Line1/Tank1/Status}, "HIGH", "status.fault", "LOW", "status.warning",  
"status.running")
```

Then: bind `meta.visible` on Label 4 to an expression `{[default]Line1/Tank1/PumpRunning}`. Toggle the tag and watch it appear and disappear. `meta.visible` being bindable is one of the most useful facts in Perspective.

**Done when:** you no longer have to think about where the binding icon is.

---

## Lab 8: The Faceplate Pattern

**The most important lab here.** *Everything professional in Perspective is a variation on this.*

**Goal:** One view that displays *any* tank, chosen by a parameter.

### Steps

1. Create a second tank: in the Tag Browser, copy `Line1/Tank1` and paste it as `Line1/Tank2`. Change `Tank2`'s `Level` to something different.
2. Create `Views/Components/TankCard`, root container **Flex**, `direction = column`.
3. Select the **View** (the top node in the Project Browser tree for this view). Go to the **PARAMS** tab. Add a parameter:
  - name: `tankPath`, direction: **input**, value: `[default]Line1/Tank1` (a default for design time)
4. Add a Label for the title. Bind `props.text` by **Expression**:

```
{view.params.tankPath}
```

(later you'll make this prettier with a `split()`; for now, seeing the path prove itself is the point.)

5. Add a Label for the level. Bind `props.text` with a **Tag binding**, and switch the binding mode to **Indirect**. Set the tag path to:

```
{1}/LevelPercent
```

Then bind reference `{1}` to `view.params.tankPath`.

6. Add a Cylindrical Tank, `props.value` → indirect `{1}/LevelPercent`, same reference.
7. Add a Label for status → indirect `{1}/Status`.
8. Save.
9. Now go back to `Pages/Overview`. Drag an **Embedded View** component onto it.
  - `props.path` = `Components/TankCard`
  - `props.params` = `{ "tankPath": "[default]Line1/Tank1" }`
10. Duplicate that Embedded View. Change the second one's param to `[default]Line1/Tank2`.
11. Preview. Two independent tank cards from **one** view definition.
12. Now change something in `TankCard` — a color, a font size. Save. Both cards change.

**Done when:** you understand that you just built a *component*, not a screen.

**Common mistakes:** putting the parameter on the root container's CUSTOM tab instead of the View's PARAMS tab (they're different nodes — click the very top item in the Project Browser for this view); leaving a trailing slash so the

indirect path becomes `[default]Line1/Tank1//LevelPercent` ; forgetting that the default param value is design-time only.

---

## Lab 9: Flex Repeater

**Goal:** Stop hardcoding the list.

### Steps

1. On `Pages/Overview`, delete the two Embedded Views from Lab 8.
2. Drop a **Flex Repeater**.
3. Set `props.path = Components/TankCard`.
4. Set `props.direction = row`, `props.wrap = wrap` so cards flow.
5. Edit `props.instances` (it's a JSON array). Type:

```
[
  { "tankPath": "[default]Line1/Tank1" },
  { "tankPath": "[default]Line1/Tank2" }
]
```

6. Preview. Two cards.
7. **Now make it dynamic.** Bind `props.instances` with a **Script** transform on an expression binding, or simpler — bind `props.instances` with an **Expression Structure** binding is fine, but the instructive version is a script. Create a Project Library script:

```
# Project Library > plant > tanks
def listInstances(folderPath):
    """Return Flex Repeater instances for every tank folder under folderPath."""
    results = system.tag.browse(folderPath, {"tagType": "Folder"})
    instances = []
    for tag in results.getResults():
        instances.append({"tankPath": str(tag["fullPath"])})
    return instances
```

8. Bind `props.instances` → **Expression** binding with value `now(10000)` (a heartbeat), then add a **Script** transform:

```
def transform(self, value, quality, timestamp):
    return plant.tanks.listInstances("[default]Line1")
```

9. Save. Now add `Line1/Tank3` in the Tag Browser (copy/paste Tank1 again).
10. Within ten seconds, a third card appears. **You did not touch the view.**

**Done when:** adding a tank to the tag tree adds a card to the screen.

**Common mistakes:** a very fast heartbeat ( `now(100)` ) hammering the Gateway — 10 s is plenty for a structure that changes monthly; forgetting `str()` around the browse result (it's a Java object, not a Python string).

---

## Lab 10: UDTs

**Goal:** Define once, stamp many.

### Steps

1. Tag Browser > right-click **Tags** > **New Tag > Data Type** (UDT Definition). Name it **Motor**.
2. Inside the definition, add members:

| Name          | Type   | Data Type |
|---------------|--------|-----------|
| Running       | Memory | Boolean   |
| Fault         | Memory | Boolean   |
| Speed         | Memory | Float     |
| SpeedSetpoint | Memory | Float     |
| RunHours      | Memory | Float     |

3. On **Speed**, enable **History**. On **Speed**, add an **Alarm**: name **High Speed**, mode **Above Setpoint**, setpoint **1800**, priority **High**. On **Fault**, add an alarm: mode **Equal**, setpoint **1** (or Bit State), priority **Critical**.
4. Add a **UDT parameter** (on the definition's parameters): **MotorName**, type String. Use it in the alarm's **Display Path** as **Line 1 / {MotorName}**.
5. Now create instances: right-click **Line1** folder > **New Tag > Data Type Instance > Motor**. Name it **Pump1**, set the **MotorName** parameter to **Pump 1**. Repeat for **Pump2**, **Pump3**.
6. Go back to the definition and **add a member** **Amps** (Memory, Float). Save.
7. Look at all three instances. They all have **Amps** now.

**Done when:** you changed one thing and three instances updated, and you can articulate why that scales to 500.

**Common mistakes:** building a UDT with no parameters (parameters are what make instances distinct beyond their name); configuring alarms on instances instead of the definition and then wondering why adding an alarm takes three hours.

Now: point your **TankCard** pattern at motors. Build **Components/MotorCard** taking a **motorPath** param, and a repeater over **[default]Line1** filtered to UDT instances.

---

## Lab 11: Named Query to Table

**Goal:** SQL the right way. (*Requires a database connection on your Gateway — if you don't have one, skip to Lab 12 and come back. For practice you can point at the Gateway's internal DB if your admin exposes one, or ask for a sandbox schema.*)

### Steps

1. Confirm a DB connection exists: Gateway **Config > Databases > Connections**, note the name.
2. Create a table and some rows (via the Designer's **Database Query Browser**, **Tools > Database Query Browser**):

```
CREATE TABLE production_log (
  id INT IDENTITY PRIMARY KEY,
  ts DATETIME,
  line_id INT,
  units INT
);
```

Insert 20 rows across a few days and two lines.

3. **Project Browser > Named Queries > New > Production/GetTotals.**

- Database: your connection
- Query type: **Query**
- Authoring tab, parameters: `startDate` (DateTime), `endDate` (DateTime)
- SQL:

```
SELECT line_id, SUM(units) AS total
FROM production_log
WHERE ts >= :startDate AND ts < :endDate
GROUP BY line_id
ORDER BY line_id
```

- Use the **Testing** tab to run it with sample values. Fix it here, not in the view.

4. New view **Pages/Reports**, root **Flex** column.

5. Add two **Date Time Input** components. Rename them `StartDate` and `EndDate` in META.

6. Add a **Table** component.

7. Bind the Table's `props.data` with a **Query** binding:

- Named Query: `Production/GetTotals`
- Parameter `startDate` → bind to `../StartDate.props.value`
- Parameter `endDate` → bind to `../EndDate.props.value`

8. Save. Preview. Change a date. The table re-queries.

**Done when:** the table reacts to the date pickers, and you didn't write a line of Python.

**Common mistakes:** binding query parameters to the wrong property path (use the property picker, don't type paths); using string concatenation to build SQL (never); forgetting that the Table wants a particular data shape — if it's blank, check the binding's return format.

## Lab 12: History and a Power Chart

**Goal:** Trends.

### Steps

1. Make a tag actually move. Either:

- add a **Programmable Device Simulator** device connection in the Gateway (**Config > Device Connections**) and create OPC tags from it, **or**

- create an Expression tag whose expression is `sin(getSecond(now(1000)) / 10.0) * 50 + 500` with history enabled — crude, but it produces a real historized curve.
2. On the tag, open **History**: enable it, pick your historian/storage provider, set `Sample Mode = On Change`, `Deadband = 1`, `Max Time Between Samples = 1 minute`.
  3. Let it run for ten minutes while you do something else.
  4. New view `Pages/Trends`. Drop a **Power Chart** component.
  5. In Preview mode, use the Power Chart's own pen-selection UI to browse to your tag and add it. Notice how much functionality you got for free — range selection, multiple axes, export.
  6. Now do it the developer-controlled way: drop a **Time Series Chart**, bind its data with a **Tag History** binding, choose your tag, set the range to "Realtime, last 1 hour", aggregation `Average`.

**Done when:** you see a curve with actual history behind it, and you understand the difference between "operator picks the pens" (Power Chart) and "developer picks the pens" (Time Series Chart).

**Common mistakes:** expecting history to exist retroactively (it starts when you enable it); a deadband so large nothing gets stored; forgetting to set a historian provider on the tag.

## Lab 13: Buttons That Do Things

**Goal:** Events, actions, and the Script Console workflow.

### Steps

1. On `Components/MotorCard` (or `TankCard`), add a **Button**, `props.text = "Start"`.
2. Event Configuration > `onActionPerformed` > add a **Script** action:

```
def runAction(self, event):
    path = self.view.params.motorPath
    system.tag.writeBlocking([path + "/Running"], [True])
```

3. Add a "Stop" button writing `False`.
4. Preview. Click. Watch the tag in the Tag Browser.
5. **Now refactor it properly.** Project Library > `plant.motors`:

```
def setRunning(motorPath, running):
    result = system.tag.writeBlocking([motorPath + "/Running"], [bool(running)])
    return result[0].good
```

Button script becomes:

```
def runAction(self, event):
    plant.motors.setRunning(self.view.params.motorPath, True)
```

6. Add a third button that uses **no script at all**: a **Set Property** action that writes `True` to `view.custom.showDetails`. Bind some component's `meta.visible` to that custom property.
7. Open `Tools > Script Console` and test your library function directly:

```
plant.motors.setRunning("[default]Line1/Pump1", True)
```

**Done when:** your button script is one line and your logic is testable in the console.

**Common mistakes:** forgetting the `def runAction(self, event):` wrapper; using `system.tag.write` (old API) instead of `writeBlocking`; doing a read right after a write and getting stale data.

---

## Lab 14: Popups with Parameters

**Goal:** Detail-on-demand.

### Steps

1. Create `Views/Popups/MotorFaceplate`, root **Flex**, param `motorPath` (input).
2. Build it out with indirect bindings (same pattern as Lab 8): speed, setpoint entry, run hours, start/stop buttons.
3. Set the view's `props.defaultSize` to something popup-shaped, e.g.  $420 \times 520$ .
4. On `MotorCard`, select the root container. Event Configuration > `onClick` > **Popup** action:
  - View: `Popups/MotorFaceplate`
  - ID: `motorFaceplate`
  - Params: `motorPath` → bind to `view.params.motorPath`
  - Title: bind to an expression using the motor name
  - Modal: true, Draggable: true
5. Add a Close button inside the faceplate with a **Popup** > **Close** action, ID `motorFaceplate`.
6. Preview. Click three different cards. Each opens the faceplate for *that* motor.

**Done when:** one popup view serves every motor.

**Common mistakes:** a different popup `id` per card (you'll stack popups); passing a literal string instead of binding the param; making the popup non-modal and then losing it behind the page.

---

## Lab 15: Message Handlers

**Goal:** Decoupled communication.

### Steps

1. In `Docks/LeftNav`, add a Button "Refresh Data".
2. `onActionPerformed` > Script action:

```
def runAction(self, event):
    system.perspective.sendMessage("refreshData", payload={}, scope="page")
```

3. In `Pages/Overview`, select the root container. Event Configuration > **Message Handlers** > add handler:
  - Message type: `refreshData`
  - Scopes: check **Page**

- Script:

```
def onMessageReceived(self, payload):  
    self.refreshBinding("props.instances")  
    system.perspective.print("Overview refreshed")
```

(`refreshBinding` takes the property path whose binding you want to re-run; adjust to match where your repeater lives — e.g. `self.getChild("FlexRepeater").refreshBinding("props.instances")`.)

4. Preview the full session (not just the view — you need the dock). Click Refresh. Check the Gateway logs / Output Console for the print.

**Done when:** a button in one view causes an action in a completely different view with no direct reference between them.

**Common mistakes:** scope mismatch (sent `page`, listening on `view`); testing in single-view preview where the dock doesn't exist; expecting `system.perspective.print` in the browser console — it goes to the Designer's Output Console / Gateway logs, because the script runs on the Gateway.

---

## Lab 16: Alarms End to End

**Goal:** From tag to operator.

### Steps

1. Confirm your `Motor` UDT alarms from Lab 10 exist. Add one to the tank: on `Line1/Tank1/LevelPercent`, alarm `High Level`, mode `Above Setpoint`, setpoint `90`, priority `High`, **Time On Delay** 5 seconds, **Display Path** `Tank 1 High Level`.
2. New view `Docks/AlarmBanner`, root Flex. Add an **Alarm Status Table**.
3. Configure it: filter to priority `High` and above; turn off columns you don't need. Set it compact.
4. Add it as a **Bottom** dock in Page Configuration Shared Settings, ID `alarmBanner`, size 120.
5. In the Tag Browser, set `Tank1/Level` to `980`. Wait five seconds.
6. The alarm appears. **Acknowledge** it from the table. Set `Level` back to `450`; watch it clear.
7. Create `Pages/AlarmHistory` with an **Alarm Journal Table**. Set its date range. Your alarm event should be in there.

**Done when:** you caused an alarm, saw it, acked it, cleared it, and found it in history.

**Common mistakes:** no alarm journal profile configured on the Gateway (journal table will be empty — check `Config > Alarming > Journal`); no time delay, producing a chattering alarm at the threshold; leaving Display Path blank so operators see raw tag paths.

---

## Lab 17: Style Classes and Theming

**Goal:** Colors in one place.

### Steps

1. `Project Browser > Perspective > Styles` > New Style Class. Create a folder `status` and classes `running`, `stopped`, `fault`, `warning`.

2. In each, set `backgroundColor`, `color`, `borderRadius`, `padding`. Make `fault` obviously red, `running` green, and so on.
3. Create `card > base`: padding, border, border-radius, a subtle shadow.
4. Go to `MotorCard`. Remove every inline `props.style.backgroundColor` you set earlier.
5. On the root container, set `props.style.classes = card.base`.
6. On the status Label, drive `props.style.classes` from state. Do it in **two steps** — this is the pattern I ship:
  - On the root container, add `custom.state` (String). Bind it with an **Indirect Tag binding** to `{1}/Status` (or build it from `Running / Fault` with an expression that references the *custom* properties you already bound, not a tag path assembled inline).
  - Bind the Label's `props.style.classes` with a **Property** binding to `view.custom.state`, then add a **Map transform**: `"FAULT" → "status.fault"`, `"RUNNING" → "status.running"`, `"STOPPED" → "status.stopped"`, with a fallback.

⚠ You cannot nest a property reference inside a tag reference in the expression language — `{{[default]{view.params.path}/Fault}}` is not valid. That is exactly what **indirect tag bindings** exist for. When you catch yourself trying to build a tag path inside `{ }`, stop and use an indirect binding instead.

7. Now change `status.fault`'s color in the Styles editor. Every card everywhere changes.
8. Add a Button somewhere with a Script action:

```
def runAction(self, event):
    current = self.session.props.theme
    self.session.props.theme = "dark" if current != "dark" else "light"
```

Click it. Check your screens still read well in both themes.

**Done when:** you can restyle the whole application from the Styles tree, and dark theme doesn't produce black-on-black anywhere.

**Common mistakes:** naming classes after colors; setting a hardcoded white background that becomes unreadable in dark theme (use theme-aware colors or leave background to the theme).

## Lab 18: Lock It Down

**Goal:** Permission that means something.

### Steps

1. Gateway `Config > Security > Users, Roles` (internal user source). Create two users: `op1` with role `Operator`, `sup1` with role `Supervisor`.
2. Make sure your project requires authentication: `Project Properties > Perspective > Permissions` — set the project to require authentication.

3. On the Start/Stop Button in `MotorCard`: Component **Permissions** — require security level `Authenticated/Roles/Supervisor`, and set the "not permitted" behavior to **Disable** (or Hide).
4. Also bind `meta.visible` on a supervisor-only section:

```
isAuthorized(false, "Authenticated/Roles/Supervisor")
```

5. **Now enforce it for real.** In the Tag Browser, open `Motor` UDT `Running` member > Security / write permissions, and require Supervisor there too.
6. Open a session, log in as `op1`. The button is gone/disabled.
7. Log in as `sup1`. It works.
8. Bonus: `Config > Security > Security Zones` — create a zone for your subnet and add it to the permission, so supervisor rights only apply from the control room.

**Done when:** the operator can't press it *and* couldn't write the tag even if they could press it.

**Common mistakes:** stopping at step 4 and calling it secure; testing with your own admin account and concluding it works.

# Part 14. Troubleshooting Playbook

---

Run these in order. It's nearly always one of the first five.

## "My binding shows nothing / the component is blank"

1. Are you in **Preview Mode**? Design mode doesn't run everything.
2. Did you **save**? Sessions show saved state.
3. Is the tag path right? Copy it from the Tag Browser rather than typing it.
4. Is the tag **quality** good? Look for the overlay. Check the tag in the Tag Browser.
5. Is the data the right **shape**? Table wants an array of objects; you may have a dataset.
6. Open the binding dialog — it shows the **current value** live. That's your debugger.

## "My script doesn't run"

1. Is the function signature exactly `def runAction(self, event):?`
2. Check the **Output Console** ( `Tools > Output Console` ) and the **Gateway logs** ( `Status > Logs` in the Gateway UI). Errors go there, not to the browser.
3. Add `system.perspective.print("here")` at the top to confirm it's firing at all.
4. Test the logic in the **Script Console** in isolation.

"I wrote a tag but the screen shows the old value" That's the write round trip. Don't read-after-write. Let the binding update. If you must confirm, check `result[0].good` from `writeBlocking`.

## "It works in the Designer but not in the session"

1. Saved?
2. Session-only properties ( `session.props.auth.*` ) are empty or different in the Designer.
3. Permissions — you're an admin in the Designer, the operator isn't.
4. Design-time parameter defaults don't apply at runtime.

## "The session is slow"

1. `Status > Perspective Sessions` in the Gateway — how many sessions, what's their load?
2. Count bindings on the offending view. Repeaters multiply them.
3. Look for script transforms doing queries or tag reads.
4. Look for `now()` with a fast poll rate.
5. Look for Query bindings with a short polling interval where a trigger would do.
6. `Status > Systems > Performance` and the Gateway logs for slow queries.

"Two people are editing and things are weird" Perspective resources lock per-editor. Coordinate. Also check that neither of you has an unsaved Designer holding a stale copy.

**Where the logs are:** Gateway web UI > `Status > Diagnostics > Logs`. Filter by logger name. When you file a question on the Inductive Automation forum, this is the first thing anyone will ask for.

## Part 15. Veteran Rules

---

Hard-won. Ignore at your own cost.

1. **Never hardcode a tag path in a reusable view.** Parameter in, indirect binding, done. The day you need the same screen for Line 2 arrives sooner than you think.
2. **UDTs before screens.** Model the plant first. Screens built on a good tag model are easy; screens built on a chaotic tag model are never good.
3. **Event scripts are three lines.** Everything real lives in the Project Library.
4. **Style classes, never inline colors.** The plant standard will change.
5. **Named Queries, never string SQL.** Not once. Not "just for testing."
6. **Flex by default.** Coordinate only when drawing a physical thing.
7. **Ask "does this need to be true with nobody looking?"** Yes → tag or Gateway event. No → binding.
8. **Every binding is a subscription.** Multiply by sessions. Design accordingly.
9. **Page navigation over view navigation.** URLs, bookmarks, and the back button are features.
10. **Set Display Path and Associated Data on every alarm.** The incident review will thank you.
11. **Turn on auditing before you need it.** You cannot retroactively audit.
12. **Export a view before a risky change.** Ten seconds of insurance.
13. **Test on the actual target device.** The 27-inch monitor lies to you about the tablet.
14. **The binding dialog's live value preview is your debugger.** Use it before you write a script.
15. **Read the manual's page for the component you're using.** Every Perspective component has a documented property list. Guessing is slower.
16. **Prefer configured actions to scripts.** Visible behavior beats hidden behavior.
17. **Don't fight the quality overlay.** It's telling you the truth.
18. **One screen, one job.** The "everything" screen is always the one operators hate.
19. **Name things for meaning, not appearance or position.** `status.fault`, not `red_label_2`.
20. **When stuck, reproduce it in a fresh throwaway view.** Half the time you find the cause while stripping it down.

## Part 16. Cheat Sheets

### Expression functions you'll use weekly

```
// logic
if(cond, a, b)                case(value, m1, r1, m2, r2, default)
coalesce(a, b)                binEnum(...)

// math
round(x, places)    abs(x)    max(a,b)    min(a,b)    sqrt(x)    pow(x,y)

// strings
concat(a, b, ...)    len(s)    substring(s, i, j)    replace(s, old, new)
toStr(x)              toInt(x) toFloat(x)            lower(s)    upper(s)
split(s, delim)       indexOf(s, sub)                trim(s)

// dates
now(pollRateMs)       dateFormat(d, "yyyy-MM-dd HH:mm:ss")
dateArithmetic(d, n, "hour")    dateDiff(d1, d2, "minute")
getHour24(d)            getDayOfWeek(d)    addDays(d, n)

// tags & security
tag("[default]Path/To/Tag")    isAuthorized(isAllOf, "Role")
hasRole("Supervisor")          runScript("plant.util.f", pollRate, arg)

// datasets
lookup(dataset, lookupValue, noMatchValue, lookupCol, resultCol)
toDataSet(...)    getKey(ds, row, col)    numRows(ds)    numCols(ds)
```

### `system.perspective.*` — the ones that matter

```
system.perspective.navigate(page="/line/3")
system.perspective.navigate(url="https://example.com", newTab=True)
system.perspective.navigate(view="Pages/Detail", params={"id": 3})

system.perspective.openPopup(id, view, params={}, title="", modal=True,
                             draggable=True, resizable=False, showCloseIcon=True)
system.perspective.closePopup(id)
system.perspective.togglePopup(id, view, ...)

system.perspective.openDock(id)
system.perspective.closeDock(id)
system.perspective.toggleDock(id)

system.perspective.sendMessage(messageType, payload={}, scope="page")

system.perspective.print(message)    # -> Gateway logs / Output Console
system.perspective.refresh()          # refresh the page
system.perspective.download(filename, data)
system.perspective.setTheme("dark")
system.perspective.login() / logout()
system.perspective.isAuthorized(isAllOf, securityLevels)
```

```
system.perspective.getSessionInfo()
system.perspective.closeSession(sessionId)
```

### **system.tag.\***

```
qvs = system.tag.readBlocking(["[default]A", "[default]B"]) # LIST in, LIST out
value, quality, ts = qvs[0].value, qvs[0].quality, qvs[0].timestamp

results = system.tag.writeBlocking(["[default]A"], [42])
ok = results[0].good

system.tag.browse("[default]Line1", {"tagType": "AtomicTag"})
system.tag.configure(basePath, tagsList, collisionPolicy) # create tags from script
system.tag.queryTagHistory(paths=..., startDate=..., endDate=..., returnSize=...)
```

### **system.db.\* and queries**

```
system.db.runNamedQuery("Path/QueryName", {"param": value})
system.db.runPrepQuery("SELECT * FROM t WHERE id = ?", [id]) # if you must
system.db.runPrepUpdate("UPDATE t SET x = ? WHERE id = ?", [x, id])
system.db.runScalarQuery("SELECT COUNT(*) FROM t")
```

### **system.dataset.\***

```
pyds = system.dataset.toPyDataSet(ds)
ds2 = system.dataset.addRow(ds, [v1, v2])
ds3 = system.dataset.filterColumns(ds, ["a", "b"])
system.dataset.toCSV(ds)
```

## **Property paths you'll type constantly**

```
view.params.<name>           # view parameter
view.custom.<name>           # view-scoped variable
this.props.<name>            # this component
../Sibling.props.text        # relative reference in bindings
session.props.auth.userName
session.props.device.type
session.custom.<name>
page.props.pageId
```

## Script object navigation

```
self                                # the component
self.parent                        # parent component
self.getSibling("Name")
self.getChild("Container").getChild("Label")
self.view                          # the view
self.view.params.x
self.view.custom.y
self.session
self.page
self.refreshBinding("props.data")
```

## Useful system tags

```
[System]Gateway/CurrentDateTime
[System]Gateway/SystemName
[System]Gateway/Performance/CPU Usage
[System]Gateway/Performance/Memory Usage
[System]Gateway/Network/Hostname
[System]Client/User/Username          (Vision-side)
```

## Part 17. Glossary

| Term                        | Meaning                                                                        |
|-----------------------------|--------------------------------------------------------------------------------|
| <b>Binding</b>              | A live link from a data source to a component property                         |
| <b>Breakpoint Container</b> | Container that swaps between two child layouts at a pixel width                |
| <b>Coordinate Container</b> | Container positioning children by x/y/w/h, fixed or percent                    |
| <b>Dataset</b>              | Ignition's native table type; what SQL and tag history return                  |
| <b>Designer</b>             | Desktop app for editing projects live on the Gateway                           |
| <b>Dock</b>                 | A view attached to a page edge (nav, header, alarm banner)                     |
| <b>Drawing Editor</b>       | 8.3's built-in vector illustration tool for custom graphics                    |
| <b>Event Stream</b>         | 8.3 resource: source → transform → handler pipeline for event data             |
| <b>Flex Container</b>       | CSS-flexbox-style container; the default choice                                |
| <b>Flex Repeater</b>        | Renders N copies of a view from an array of parameter objects                  |
| <b>Gateway</b>              | The server: runs everything, holds the tag system, serves sessions             |
| <b>Historian</b>            | The subsystem that stores tag values over time                                 |
| <b>IdP</b>                  | Identity Provider — where users authenticate                                   |
| <b>Indirect tag binding</b> | Tag binding whose path contains <code>{1}</code> -style placeholders           |
| <b>Jython</b>               | Python 2.7 on the JVM; Ignition's scripting language                           |
| <b>Named Query</b>          | Stored, parameterized, permission-checked SQL                                  |
| <b>Page Configuration</b>   | Mapping of URLs to primary views, plus dock setup                              |
| <b>Perspective</b>          | The web/mobile visualization module                                            |
| <b>Project Library</b>      | Project-scoped Python modules, importable anywhere in the project              |
| <b>Quality</b>              | Metadata on every tag value: Good / Bad / Uncertain and why                    |
| <b>Security Level</b>       | Hierarchical permission node, e.g. <code>Authenticated/Roles/Supervisor</code> |
| <b>Security Zone</b>        | Permission based on <i>where</i> the request originates                        |
| <b>Session</b>              | One user's running instance of a Perspective project                           |
| <b>Style Class</b>          | Named, reusable set of CSS properties                                          |
| <b>Tag</b>                  | A named value in the Gateway with quality, timestamp, alarms, history          |
| <b>Tag Provider</b>         | A namespace of tags, e.g. <code>default</code> , <code>System</code>           |
| <b>Theme</b>                | Gateway-level palette; <code>light</code> , <code>dark</code> , or custom      |
| <b>Transform</b>            | A step that reshapes a binding's value: Map, Format, Expression, Script        |
| <b>UDT</b>                  | User Defined Type — a reusable tag structure. Your biggest leverage            |

| Term          | Meaning                                                 |
|---------------|---------------------------------------------------------|
| <b>View</b>   | The unit of Perspective design; reusable, parameterized |
| <b>Vision</b> | The older Java desktop-client visualization module      |

# Part 18. Self Assessment

---

Answer without looking. If you can't, the linked section is your homework.

## Architecture

1. Where does a Perspective event script execute, and why does that matter? → [1.1](#)
2. Trace what happens between an operator's click and the PLC receiving a value. → [1.5](#)
3. What's the practical difference between Perspective and Vision? → [1.2](#)

**Layout** 4. Given a header, a scrollable body, and a footer — what container and what position props? → [3.2](#) 5. When is Coordinate the right answer? → [3.2](#)

**Data binding** 6. You need one screen to work for 200 motors. Describe the mechanism. → [Lab 8](#) 7. Name the four transform types and give a use for each. → [4.3](#) 8. Expression binding vs expression tag — how do you decide? → [4.5](#)

**Reuse** 9. What does `props.instances` on a Flex Repeater take? → [3.4](#) 10. Why are UDTs the highest-leverage feature in Ignition? → [1.4](#)

**SQL** 11. Three reasons for Named Queries over inline SQL. → [6.3](#) 12. What is a Literal parameter and why is it dangerous? → [6.3](#)

**Scripting** 13. What's in scope inside `def runAction(self, event): ?` → [5.2](#) 14. Two views can't reference each other. How do they communicate? → [5.4](#) 15. Why should event scripts be three lines? → [5.3](#)

**Security** 16. Why is hiding a button not security, and what's the complete answer? → [9.2](#)

**8.3** 17. Name three things 8.3 added and why each matters. → [Part 11](#)

# Where to Go Next

---

- **Inductive University** ( [inductiveuniversity.com](https://inductiveuniversity.com) ) — free, official, short videos with a version selector. Set it to 8.3. Do the Perspective track; it pairs well with this guide's labs.
- **The user manual** ( [docs.inductiveautomation.com](https://docs.inductiveautomation.com) , version selector → **8.3** ) — the appendix has a page per component listing every property. Bookmark the Perspective section.
- **The forum** ( [forum.inductiveautomation.com](https://forum.inductiveautomation.com) ) — unusually high signal. Search before asking; when you ask, include your version, the exact error, and the Gateway log excerpt.
- **8.3 release notes** ( [inductiveautomation.com/downloads/releasenotes/8.3.x](https://inductiveautomation.com/downloads/releasenotes/8.3.x) ) — read the notes between 8.3.0 and 8.3.9 once, so you know what changed under you.
- **Ignition Exchange** — community resources and reusable Perspective components you can import and read. Reading other people's views is a fast way to level up.

After the capstone, the natural next topics are: project inheritance for multi-site work, the Reporting module, alarm notification pipelines, Event Streams (8.3), MQTT/Sparkplug for distributed architectures, and redundancy.

---

*Built for Ignition 8.3.9. When the screen disagrees with this document, believe the screen — then go read the manual page for whatever surprised you. That habit is the actual skill.*